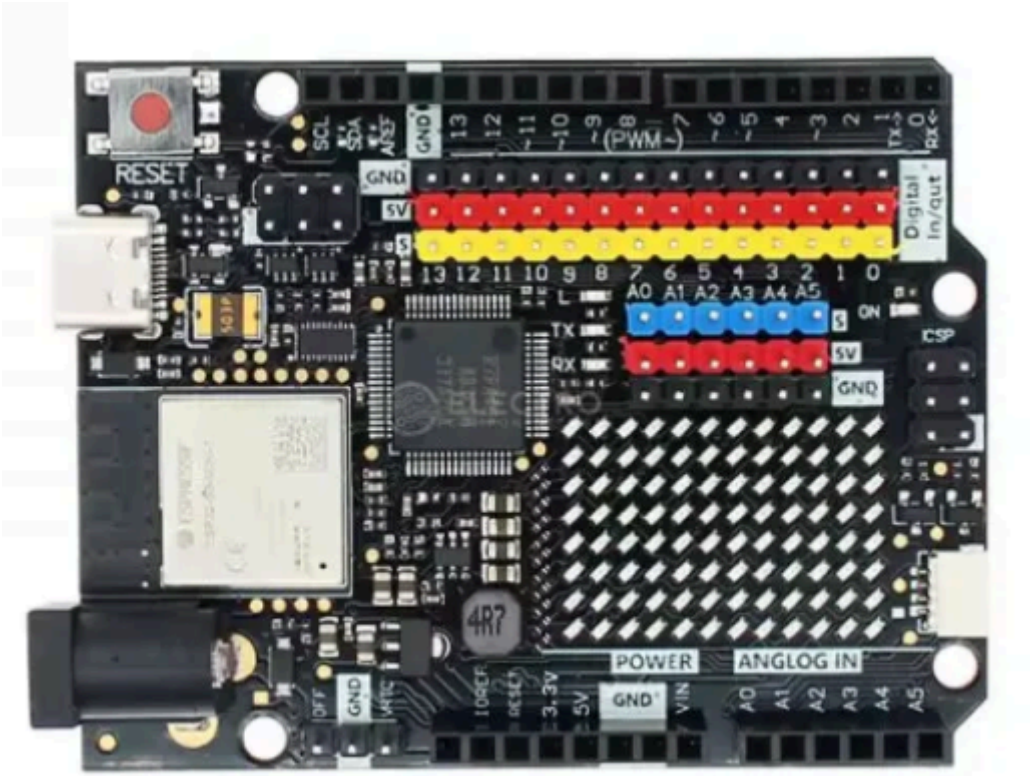
El port sèrie



Seba Parvu & Iker Miranda
26/02/2026
Proje. Inter.
Dani Soldevila

INDEX

El port sèrie 1
1. Objectiu de la pràctica 2
2. Explicació de conceptes clau 2
3. Esquema de connexions (fritzing) 3
4. Diagrama de flux del programa (com fer un diagrama de flux) 4
5. Flux de dades i procés de lectura/escriptura 5
6. Proves 6
7. Explicació detallada del seguiment del projecte dins de Github. 9

1. Objectiu de la pràctica

- Breu descripció de la pràctica

L'objectiu d'aquesta pràctica és aprendre a comunicar-se amb l'ESP32 utilitzant el Monitor Sèrie, llegint dades enviades per l'usuari, processant-les amb funcions com trim() i equalsIgnoreCase(), i enviant respostes amb Serial.println(). També serveix per controlar un LED segons les comandes i entendre el flux de dades: Monitor Sèrie → ESP32 → LED → Monitor Sèrie. A més, la pràctica introdueix conceptes bàsics de programació com classes, objectes i mètodes, i relaciona el codi amb l'esquema físic del circuit i el diagrama de flux del programa.

2. Explicació de conceptes clau

- Què és una classe, un objecte i un mètode.

-Una classe és com una recepta o un model. Explica com ha de ser una cosa i què pot fer, però no fa res tota sola.

-Un objecte és la cosa real creada a partir de la classe. És el que utilitza el programa per fer accions.

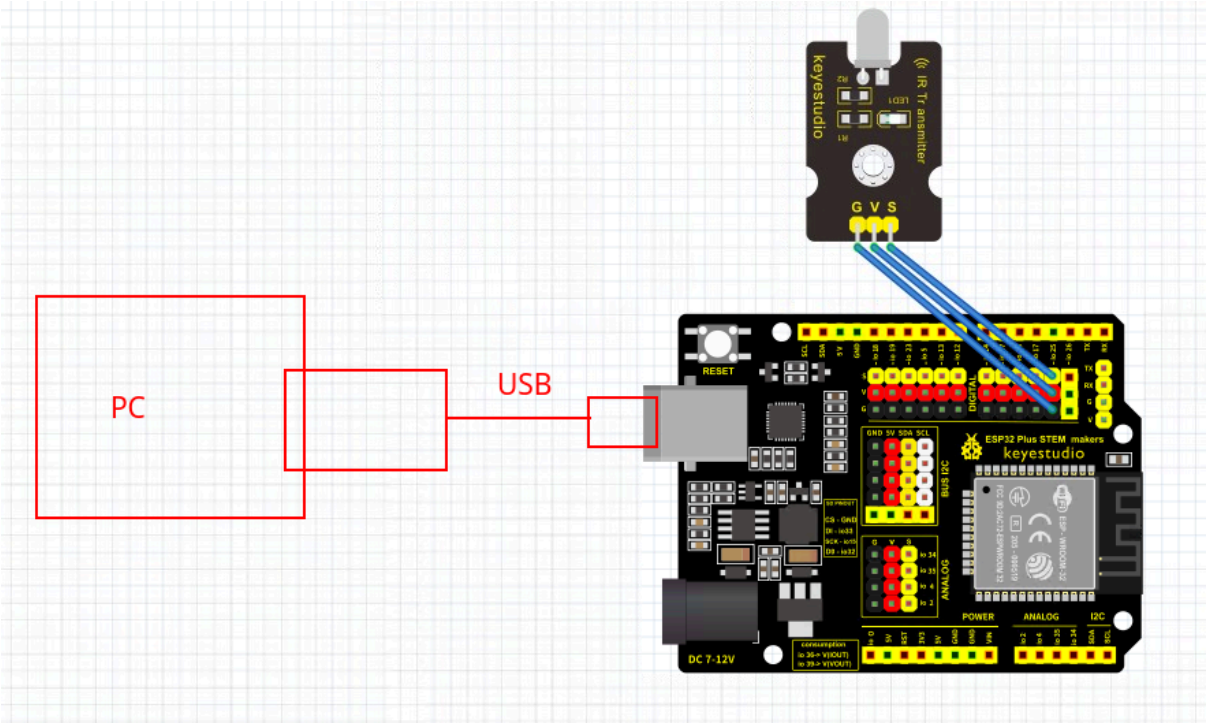
-Un mètode és una acció que pot fer un objecte. Per exemple: escriure un missatge o llegir dades.

- Relació amb l'objecte `Serial`.

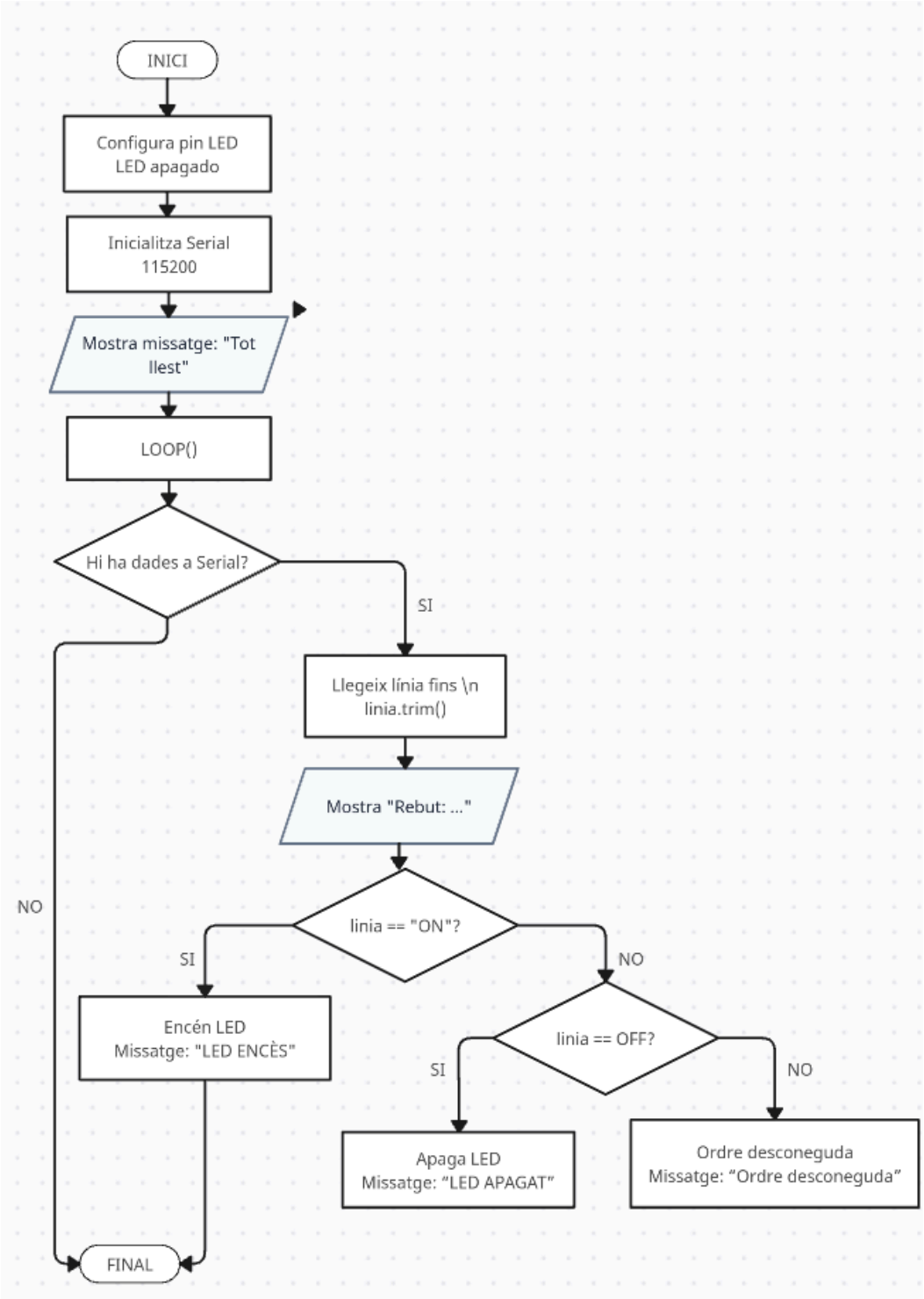
Serial és un objecte que serveix per comunicar l'ESP32 amb l'ordinador.

Quan escrivim Serial.begin(), Serial.print() o Serial.readStringUntil(), estem dient a l'objecte Serial que faci alguna acció, és a dir, estem usant els seus mètodes.

3. Esquema de connexions (fritzing)



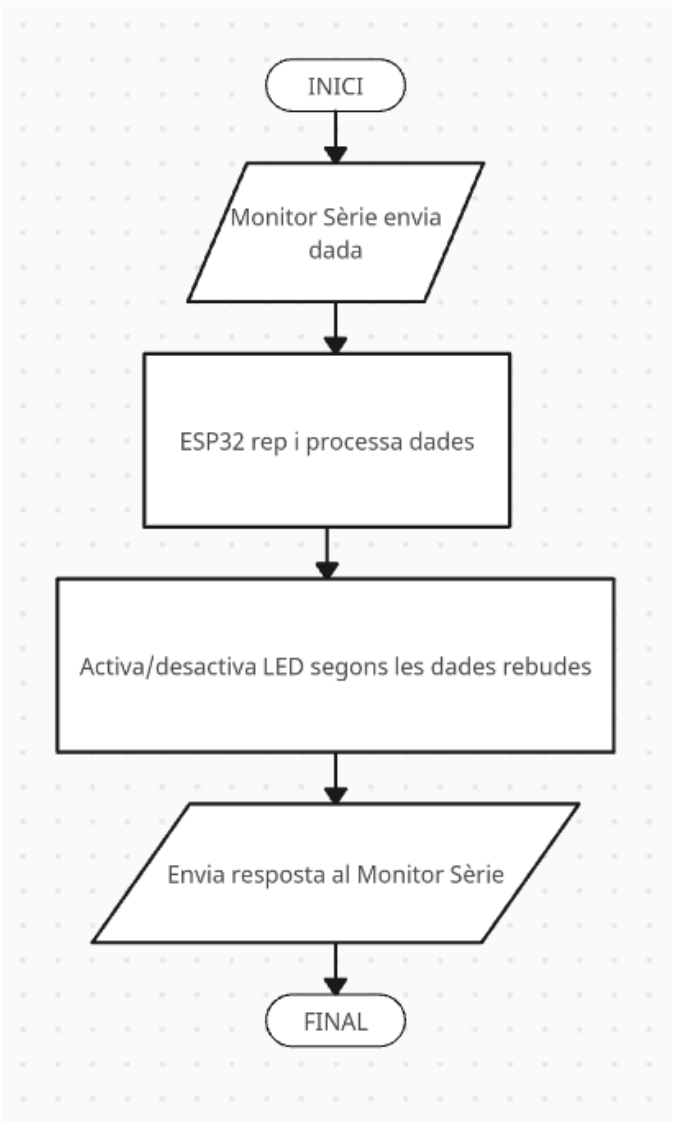
4. Diagrama de flux del programa (com fer un diagrama de flux)



https://app.creately.com/d/D6fiOiKyOyn/edit

5. Flux de dades i procés de lectura/escriptura

- Diagrama visual del flux de dades: Monitor Sèrie → ESP32 → LED / Monitor Sèrie.



https://app.creately.com/d/XCCmEIIfzjy/edit

- Explicació de com es llegeixen dades (`Serial.available()`, `readStringUntil()`), com es processen (`trim()`, `equalsIgnoreCase()`) i com s'envien respostes (`println()`).

Com llegir les dades:

Per llegir dades al ESP32, primer hem de comprovar si l'usuari ha enviat alguna cosa pel Monitor Sèrie. Això ho fem amb Serial.available(), que ens diu si hi ha bytes disponibles per llegir.
Si hi ha dades, les llegim amb readStringUntil('\n'), que agafa tot el text que l'usuari ha escrit fins que prem Enter. Així obtenim la comanda completa i la guardem en una variable per processar-la després.

Com processar les dades:

Un cop tenim el text rebut, és important netejar-lo amb trim(). Aquesta funció elimina tots els espais en blanc o salts de línia que hi pugui haver al principi o al final del text, evitant errors quan comparem la comanda.
Després, per decidir quina acció cal fer, utilitzem equalsIgnoreCase(). Aquesta funció compara el text amb la comanda esperada sense tenir en compte si està en majúscules o minúscules. Per exemple, "ON", "on" o "On" serien considerades iguals.

Com enviar respostes:

Finalment, un cop hem processat la comanda i hem fet l'acció corresponent (per exemple encendre o apagar un LED), enviem una resposta al Monitor Sèrie amb Serial.println().
Això permet que l'usuari vegi què ha passat. Per exemple, podem mostrar missatges com "LED encès" quan activem el LED o "LED apagat" quan el desactivem. D'aquesta manera, el programa comunica de manera clara què està fent segons les dades rebudes.

6. Proves

Vídeo demostratiu del funcionament

Els codis han de iniciar-se amb 115200 de velocitat del port sèrie.

EXECUCIÓ CODI ISSUE 1:

```
ÿÿets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4980
load:0x40078000,len:16612
load:0x40080400,len:3480
entry 0x400805b4
Tot llest!
```

EXECUCIÓ CODI ISSUE 2:

```
/dev/ttyUSB0

ets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4980
load:0x40078000,len:16612
load:0x40080400,len:3480
entry 0x400805b4
Issue 2 llest. Escriu alguna cosa i prem ENTER.
Hola
```

EXECUCIÓ CODI ISSUE 3:

```
/dev/ttyUSB0

ets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4980
load:0x40078000,len:16612
load:0x40080400,len:3480
entry 0x400805b4
Issue 3 llest. Escriu text i es mostrarà amb prefix.
Rebut: Hola bona nit
```

EXECUCIÓ CODI ISSUE 4:

```
/dev/ttyUSB0

ÿÿets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4980
load:0x40078000,len:16612
load:0x40080400,len:3480
entry 0x400805b4
Issue 4 llest. Escriu ON o OFF per controlar el LED.
Rebut: ON
LED ENCÈS
Rebut: OFF
LED APAGAT
Rebut: ON
```

EXECUCIÓ CODI COMPLET JUNTANT ELS 4 ISSUES:

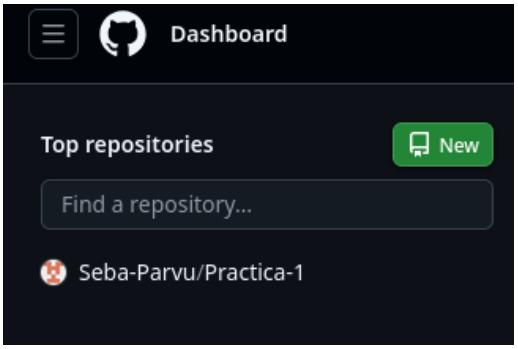
```
/dev/ttyUSB0

ets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4980
load:0x40078000,len:16612
load:0x40080400,len:3480
entry 0x400805b4
Tot llest
Rebut: ON
LED ENCÈS
Rebut: OFF
LED APAGAT
Rebut: ON
```

```
Rebut: asd
Ordre desconeguda
```

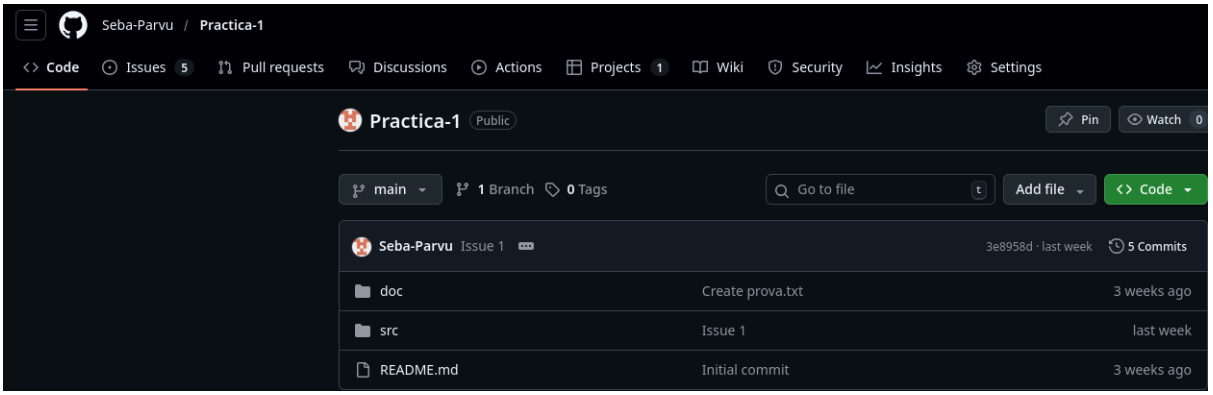
VIDEO:
https://drive.google.com/file/d/1o6n2qTNOEfz1KVl3eLQKQ74QSlkE4rof/view?usp=drive_link

7. Explicació detallada del seguiment del projecte dins de Github. Amb imatges reals del vostre projecte a GitHub cal explicar tot el procés (repositori, taulell Kanban, etc…)

Aquest projecte s'organitzarà amb GitHub



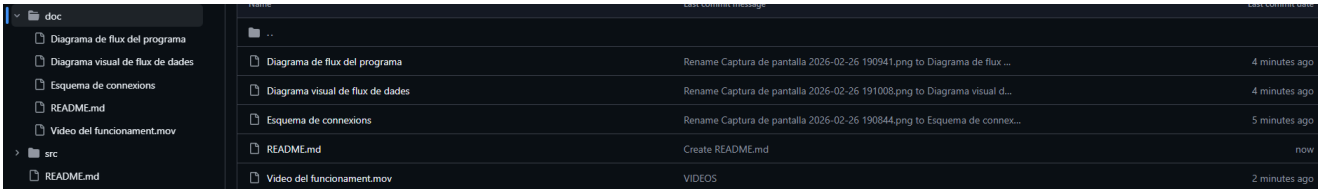
Has de tenir un repositori per guardar els arxius, els issues i els projectes a part d'altres coses que per ara no hem utilitzat



Entra al repositori. Al entrar veures que pots crear fitxers i carpetes.

Has de crear 2 carpetes: doc i src

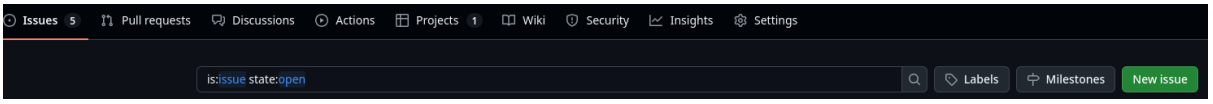
Per explicar el projecte també pots afegir un readme



A la carpeta doc hi ha d'haver les imatges i videos que hem d'utilitzar per la documentació. També ha de estar el pdf amb la documentació.

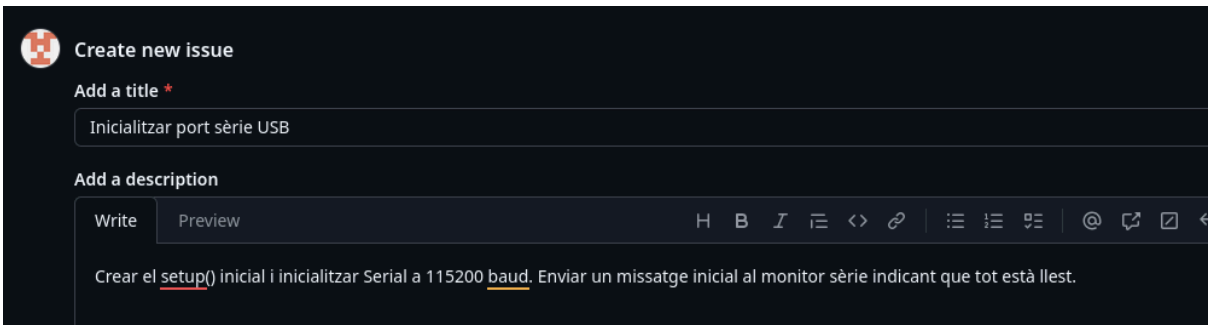
A la carpeta src hi ha d'haver els codis que hem utilitzat per solucionar els issues i a part el codi complet que funciona amb tot el demanat.

A aquestes 2 carpetes pots afegir un readme per explicar per a que serveix la carpeta

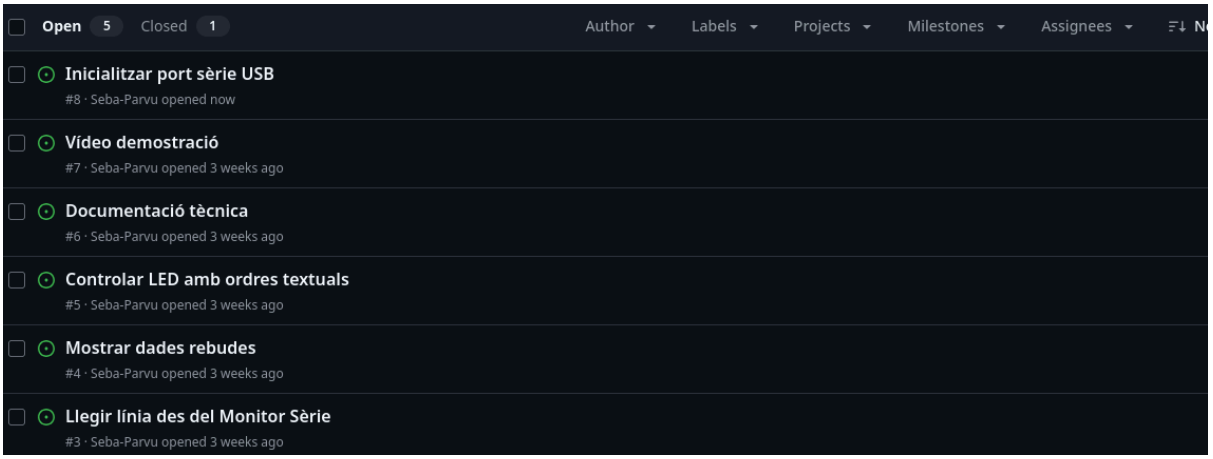


Despres als issues i comença a crear el primer issue(són 6 en total)

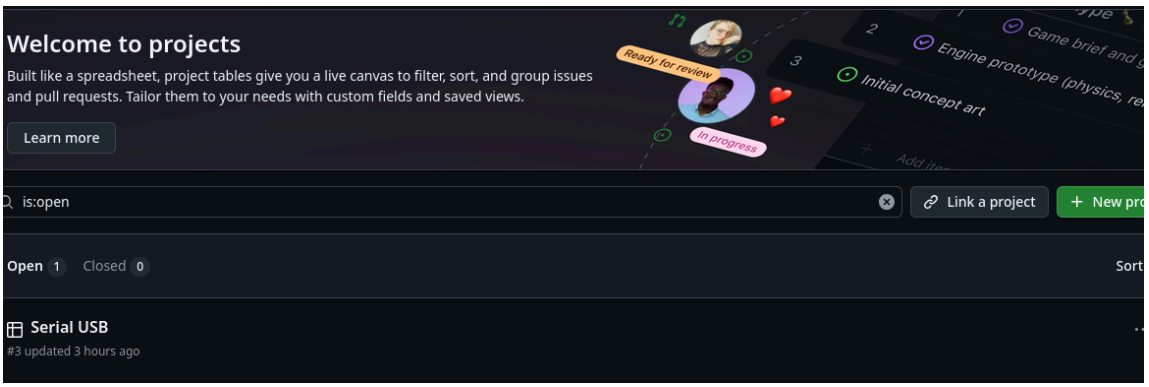
Un issue a GitHub és un tiquet per informar d'un problema, proposar una millora o debatre una tasca en un projecte.



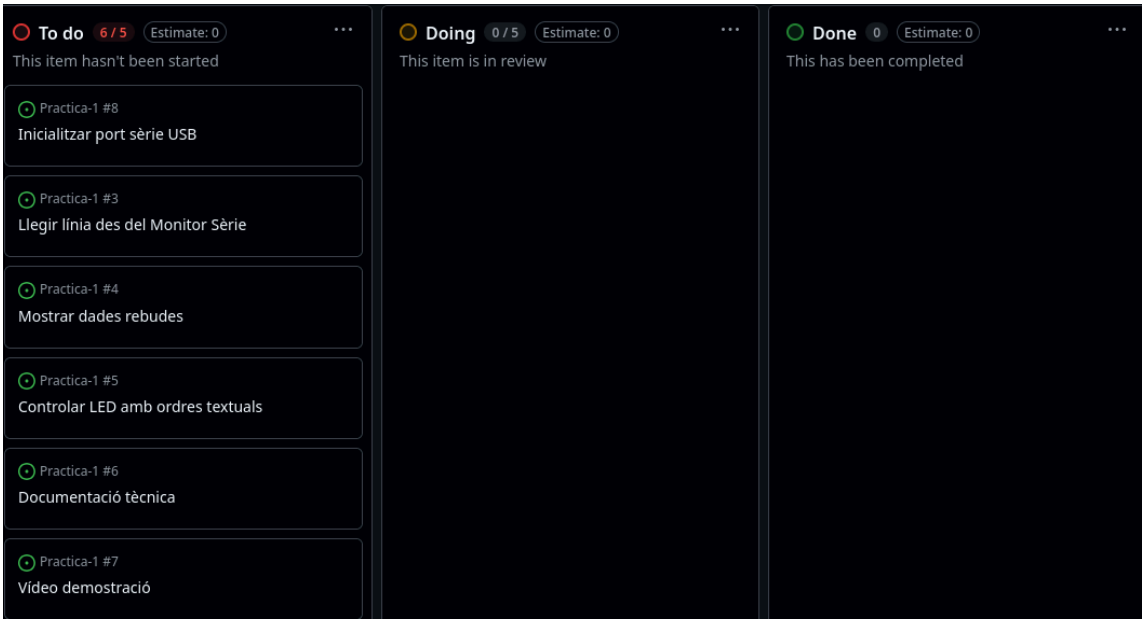
Posa el nom del issue i la seva descripció



Aquí es veuen tots els issues creat



Ara entra a projects i crea el teu projecte



Al entrar al projecte has de configurar-lo amb una plantilla Kanban.

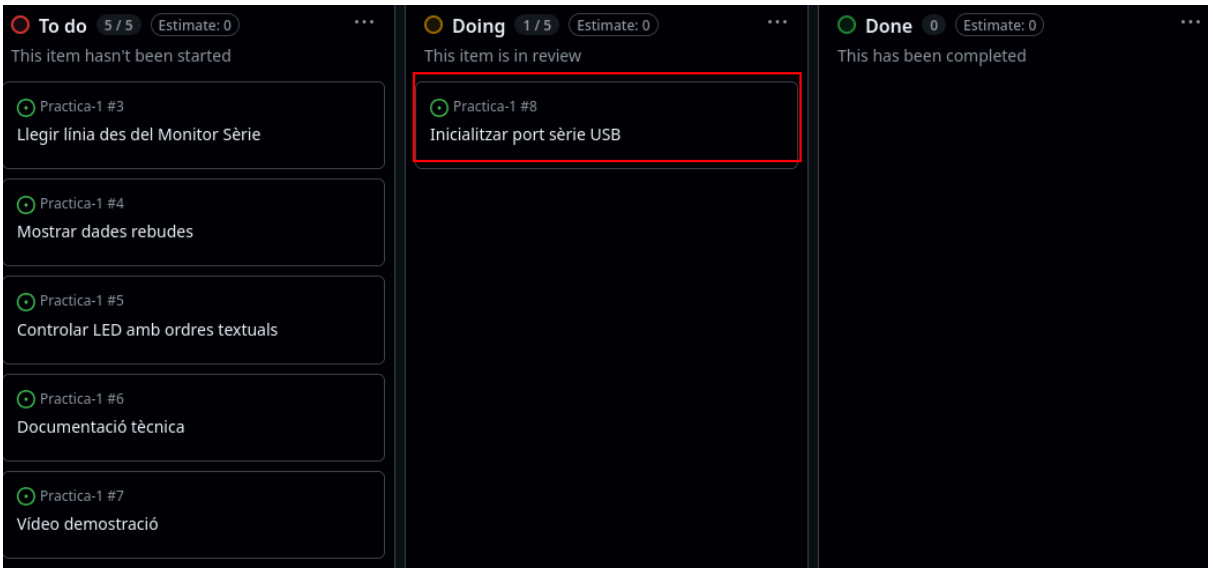
Quan ho configuris haures de posar 3 apartats:

To do: tasques que s'han de començar

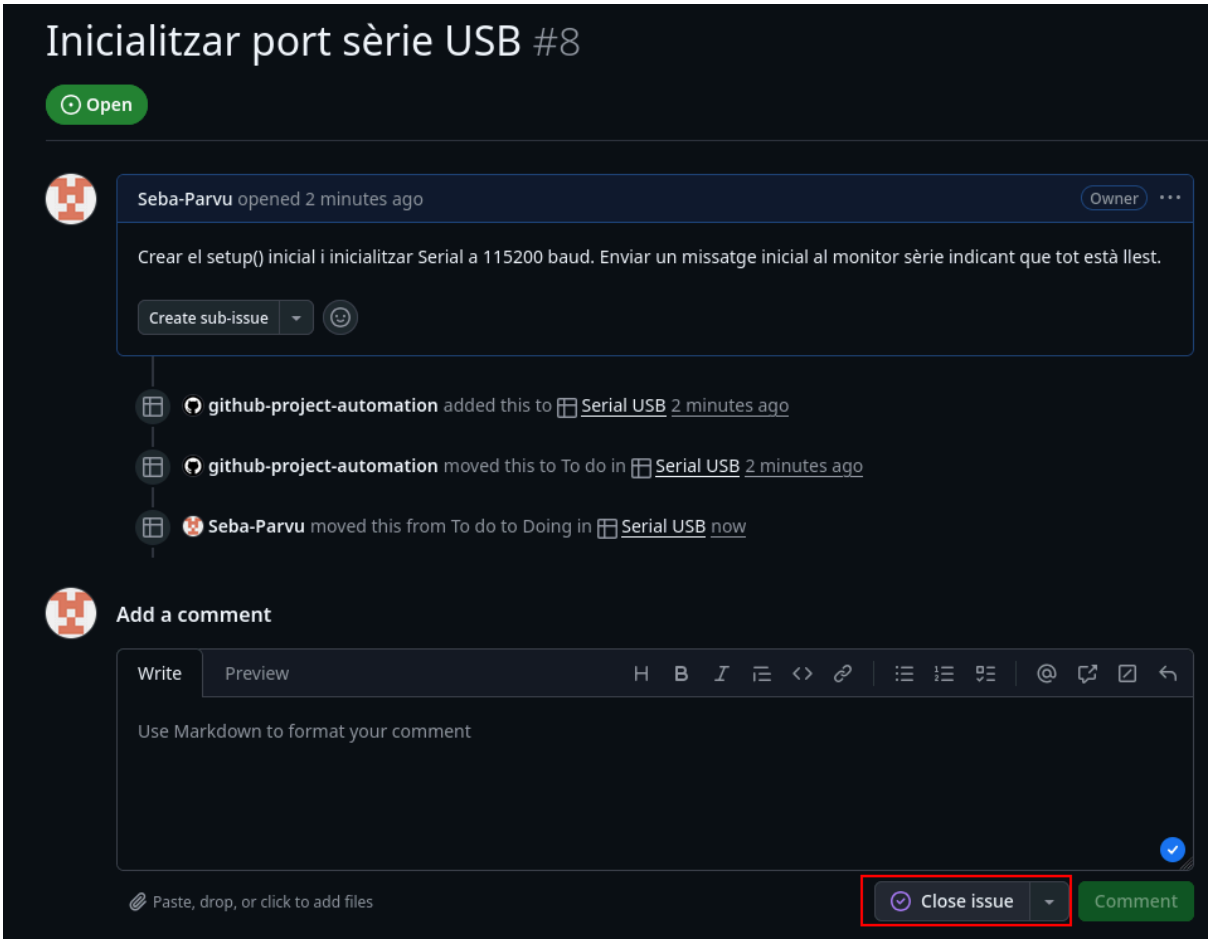
Doing: tasques que s'estén fent

Done: Tasques que s'han acabat de fer

Quan posis aquests apartats has de veure que els issues estiguin al "To do", o sigui que s'han de començar a fer.

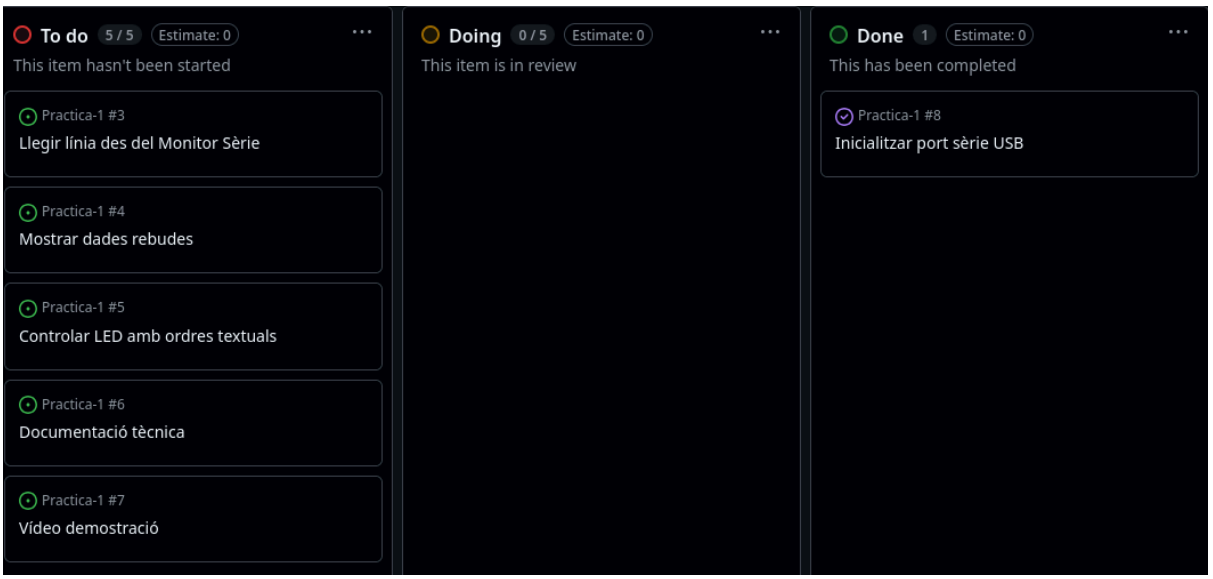


Quan comences a fer un issue has de moure'l de "To do" al "Doing", per marcar que l'has començat a solucionar.



Després quan has acabat de solucionar el issue l'has de tancar per posar-lo a "Done".

Entra a la configuració del issue i tanca'l



Una vegada tancat surt el issue a "Done"